

Compiler Construction

Teacher : Zulfiqar Ali

Email: Zulfiqar.rafique@nu.edu.pk

Office: No.6,Basement -1 (CS Block 1)

Week 8:

More Powerful LR Parsers

CLR(1) - Canonical LR

LALR(1)

More Powerful LR Parsers

In this section, we shall extend the previous LR parsing techniques to use one symbol of lookahead on the input. There are two different methods:

1. The “canonical-LR” or just “LR” method, which makes full use of the lookahead symbol(s). This method uses a large set of items, called the LR(1) items.
2. The “lookahead-LR” or “LALR” method, which is based on the LR(0) sets of items, and has many fewer states than typical parsers based on the LR(1) items. By carefully introducing lookaheads into the LR(0) items, we can handle many more grammars with the LALR method than with the SLR method, and build parsing tables that are no bigger than the SLR tables. LALR is the method of choice in most situations.

Canonical LR(1)

We shall now present the most general technique for constructing an LR parsing table from a grammar. Recall that in the SLR method, state i calls for reduction by $A \rightarrow \alpha$ if the set of items I_i contains item $[A \rightarrow \alpha \cdot]$ and input symbol a is in $\text{FOLLOW}(A)$. In some situations, however, when state i appears on top of the stack, the viable prefix $\beta\alpha$ on the stack is such that βA cannot be followed by a in any right-sentential form. Thus, the reduction by $A \rightarrow \alpha$ should be invalid on input a .

It is possible to carry more information in the state that will allow us to rule out some of these invalid reductions by $A \rightarrow \alpha$. By splitting states when necessary, we can arrange to have each state of an LR parser indicate exactly which input symbols can follow a handle α for which there is a possible reduction to A .

Canonical LR(1)

The extra information is incorporated into the state by redefining items to include a terminal symbol as a second component. The general form of an item becomes $[A \rightarrow \alpha \cdot \beta, a]$, where $A \rightarrow \alpha\beta$ is a production and a is a terminal or the right endmarker $\$$. We call such an object an *LR(1) item*. The 1 refers to the length of the second component, called the *lookahead* of the item.⁶ The lookahead has no effect in an item of the form $[A \rightarrow \alpha \cdot \beta, a]$, where β is not ϵ , but an item of the form $[A \rightarrow \alpha \cdot, a]$ calls for a reduction by $A \rightarrow \alpha$ only if the next input symbol is a . Thus, we are compelled to reduce by $A \rightarrow \alpha$ only on those input symbols a for which $[A \rightarrow \alpha \cdot, a]$ is an LR(1) item in the state on top of the stack. The set of such a 's will always be a subset of $\text{FOLLOW}(A)$, but it could be a proper subset, as in Example 4.51.

Formally, we say LR(1) item $[A \rightarrow \alpha \cdot \beta, a]$ is *valid* for a viable prefix γ if there is a derivation $S \xRightarrow{*}_{rm} \delta A w \xRightarrow{rm} \delta \alpha \beta w$, where

1. $\gamma = \delta \alpha$, and
2. Either a is the first symbol of w , or w is ϵ and a is $\$$.

Example Grammar: $S \rightarrow B B$
 $B \rightarrow a B \mid b$

The method for building the collection of sets of valid LR(1) items is essentially the same as the one for building the canonical collection of sets of LR(0) items. We need only to modify the two procedures CLOSURE and GOTO.

```
SetOfItems CLOSURE( $I$ ) {  
    repeat  
        for ( each item  $[A \rightarrow \alpha \cdot B \beta, a]$  in  $I$  )  
            for ( each production  $B \rightarrow \gamma$  in  $G'$  )  
                for ( each terminal  $b$  in FIRST( $\beta a$ ) )  
                    add  $[B \rightarrow \cdot \gamma, b]$  to set  $I$ ;  
    until no more items are added to  $I$ ;  
    return  $I$ ;  
}
```

```

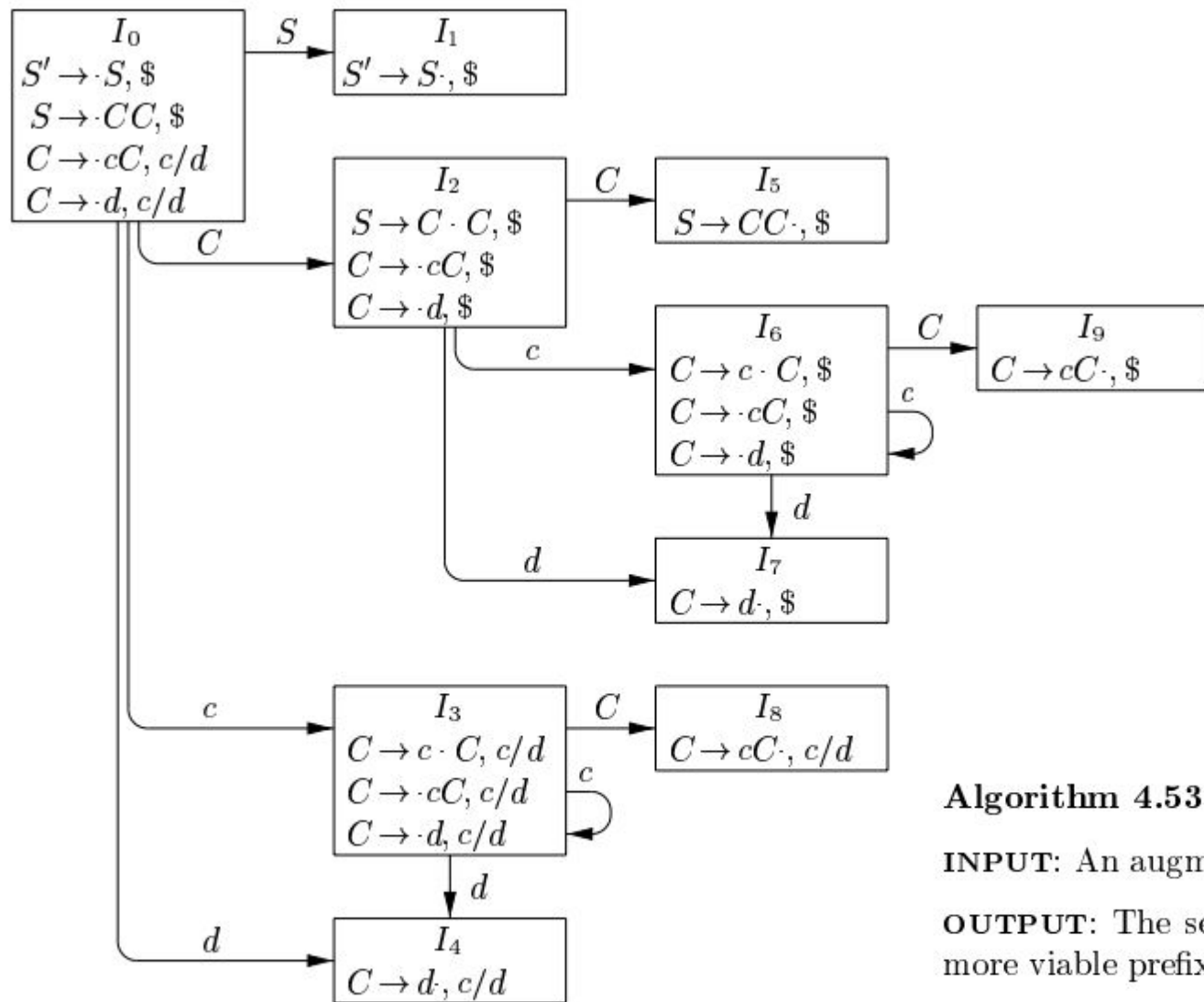
SetOfItems GOTO( $I, X$ ) {
    initialize  $J$  to be the empty set;
    for ( each item  $[A \rightarrow \alpha \cdot X \beta, a]$  in  $I$  )
        add item  $[A \rightarrow \alpha X \cdot \beta, a]$  to set  $J$ ;
    return CLOSURE( $J$ );
}

void items( $G'$ ) {
    initialize  $C$  to  $\{\text{CLOSURE}(\{[S' \rightarrow \cdot S, \$])\}\}$ ;
    repeat
        for ( each set of items  $I$  in  $C$  )
            for ( each grammar symbol  $X$  )
                if ( GOTO( $I, X$ ) is not empty and not in  $C$  )
                    add GOTO( $I, X$ ) to  $C$ ;
    until no new sets of items are added to  $C$ ;
}

```

Figure 4.40: Sets-of-LR(1)-items construction for grammar G'

To appreciate the new definition of the CLOSURE operation, in particular, why b must be in $\text{FIRST}(\beta a)$, consider an item of the form $[A \rightarrow \alpha \cdot B \beta, a]$ in the set of items valid for some viable prefix γ . Then there is a rightmost derivation $S \xRightarrow{*}_{rm} \delta A a x \Rightarrow \delta \alpha B \beta a x$, where $\gamma = \delta \alpha$. Suppose $\beta a x$ derives terminal string by . Then for each production of the form $B \rightarrow \eta$ for some η , we have derivation $S \xRightarrow{*}_{rm} \gamma B b y \Rightarrow \gamma \eta b y$. Thus, $[B \rightarrow \cdot \eta, b]$ is valid for γ . Note that b can be the first terminal derived from β , or it is possible that β derives ϵ in the derivation $\beta a x \xRightarrow{*}_{rm} by$, and b can therefore be a . To summarize both possibilities we say that b can be any terminal in $\text{FIRST}(\beta a x)$, where FIRST is the function from Section 4.4. Note that x cannot contain the first terminal of by , so $\text{FIRST}(\beta a x) = \text{FIRST}(\beta a)$. We now give the LR(1) sets of items construction.



Algorithm 4.53: Construction of the sets of LR(1) items.

INPUT: An augmented grammar G' .

OUTPUT: The sets of LR(1) items that are the set of items valid for one or more viable prefixes of G' .

Figure 4.41: The GOTO graph for grammar (4.55)

Canonical LR(1) Parsing Table

We now give the rules for constructing the LR(1) ACTION and GOTO functions from the sets of LR(1) items. These functions are represented by a table, as before. The only difference is in the values of the entries.

Algorithm 4.56: Construction of canonical-LR parsing tables.

INPUT: An augmented grammar G' .

OUTPUT: The canonical-LR parsing table functions ACTION and GOTO for G' .

METHOD:

1. Construct $C' = \{I_0, I_1, \dots, I_n\}$, the collection of sets of LR(1) items for G' .
2. State i of the parser is constructed from I_i . The parsing action for state i is determined as follows.
 - (a) If $[A \rightarrow \alpha \cdot a \beta, b]$ is in I_i and $\text{GOTO}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to “shift j .” Here a must be a terminal.
 - (b) If $[A \rightarrow \alpha \cdot, a]$ is in I_i , $A \neq S'$, then set $\text{ACTION}[i, a]$ to “reduce $A \rightarrow \alpha$.”
 - (c) If $[S' \rightarrow S \cdot, \$]$ is in I_i , then set $\text{ACTION}[i, \$]$ to “accept.”

If any conflicting actions result from the above rules, we say the grammar is not LR(1). The algorithm fails to produce a parser in this case.

3. The goto transitions for state i are constructed for all nonterminals A using the rule: If $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}[i, A] = j$.
4. All entries not defined by rules (2) and (3) are made “error.”
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow \cdot S, \$]$.

The table formed from the parsing action and goto functions produced by Algorithm 4.56 is called the *canonical* LR(1) parsing table. An LR parser using this table is called a canonical-LR(1) parser. If the parsing action function has no multiply defined entries, then the given grammar is called an *LR(1) grammar*. As before, we omit the “(1)” if it is understood.

Example 4.57: The canonical parsing table for grammar (4.55) is shown in Fig. 4.42. Productions 1, 2, and 3 are $S \rightarrow CC$, $C \rightarrow cC$, and $C \rightarrow d$, respectively. $\square$

Every SLR(1) grammar is an LR(1) grammar, but for an SLR(1) grammar the canonical LR parser may have more states than the SLR parser for the same grammar. The grammar of the previous examples is SLR and has an SLR parser with seven states, compared with the ten of Fig. 4.42.

STATE	ACTION			GOTO	
	<i>c</i>	<i>d</i>	\$	<i>S</i>	<i>C</i>
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Figure 4.42: Canonical parsing table for grammar (4.55)

Example: 1

CLR(1) or LR(1).

Exmpl

Grammar

$X \rightarrow YY$

$Y \rightarrow aY$

$Y \rightarrow b$

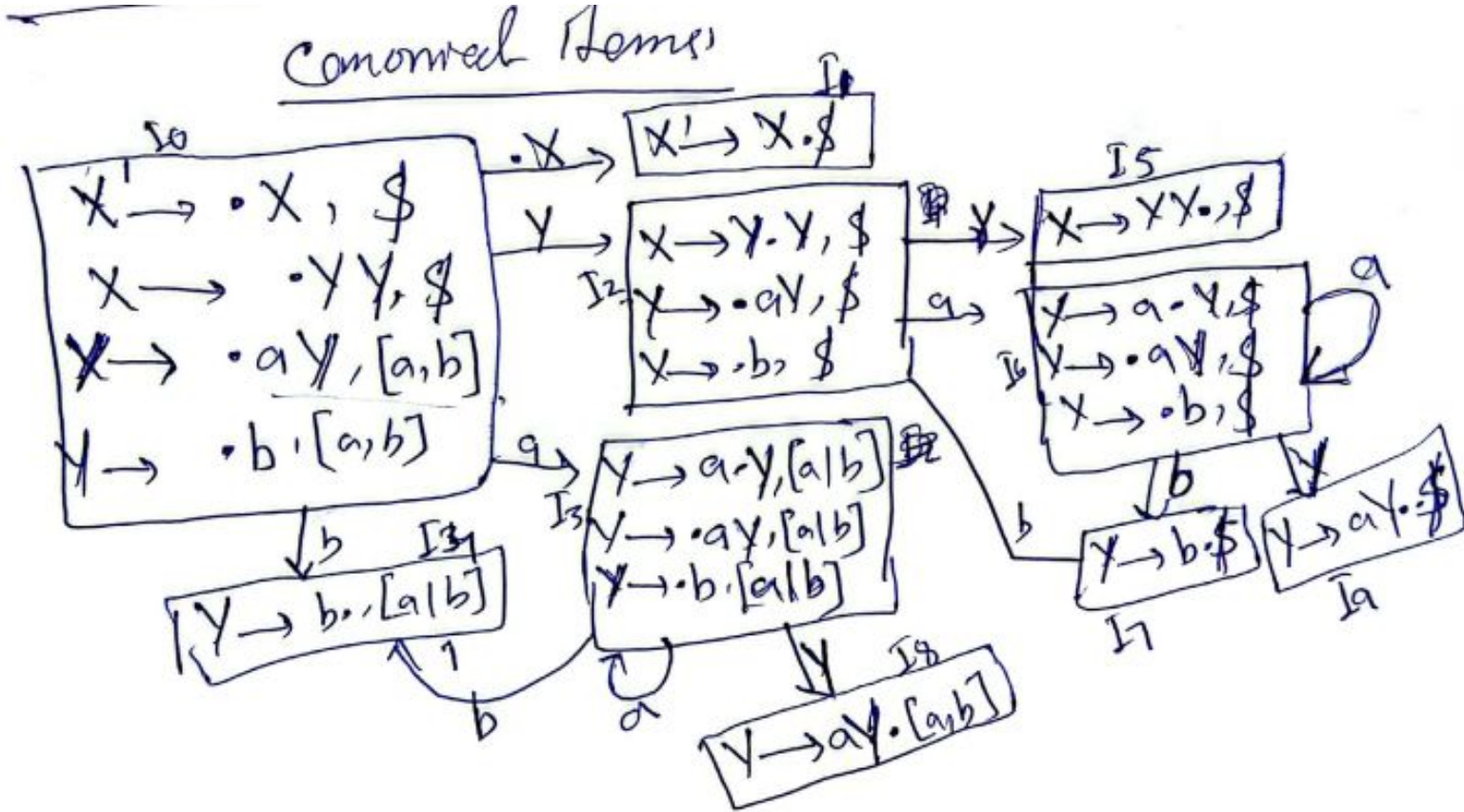
Augmented Grammar

$X' \rightarrow \bullet X \rightarrow \circ$

$Y \rightarrow \bullet aY \rightarrow \textcircled{2}$

$\left| \begin{array}{l} X \rightarrow \bullet YY \rightarrow \textcircled{1} \\ Y \rightarrow \bullet b \rightarrow \textcircled{3} \end{array} \right.$

Canonical Items



more states in LR(1)

CLR Parsing Table

States	Actions			goto	
	a	b	\$	x	y
I ₀	r s3	r s4	r	r 1	r 2
I ₁	s		Accept	1	
I ₂	s6	s7			5
I ₃	s3	s4			8
I ₄	R3	R3			
I ₅			R1		
I ₆	s6	s7			9
I ₇			R3		
I ₈	R2	R2			
I ₉			R2		

Stack	Input	Action
0	abab\$	S3
0a3	bab\$	S2
0a3b2	ab\$	R3: $Y \rightarrow b$
0a3Y8	ab\$	R2: $Y \rightarrow aY$
0Y4	ab\$	S6
0Y4ab	b\$	S7
0Y4abb7	\$	R3: $Y \rightarrow b$
0Y4abY9	\$	R2: $Y \rightarrow aY$
0Y4Y5	\$	R1: $X \rightarrow YY$
0X1	\$	ACCEPT

Example 2: CLR(1)

Grammar

CLR(1)
Canonical LR(1).

$S \rightarrow aAd / bBd / aBe / bAe$

$A \rightarrow c$

$B \rightarrow c$

Augmented Grammar

$S' \rightarrow S \rightarrow 0$

$S \rightarrow aAd \rightarrow 1$

$S \rightarrow bBd \rightarrow 2$

$S \rightarrow aBe \rightarrow 3$

$S \rightarrow bAe \rightarrow 4$

$A \rightarrow c \rightarrow 5$

$B \rightarrow c \rightarrow 6$

$S \rightarrow aAd$

Augmented Grammar

$S' \rightarrow S \rightarrow \epsilon$

$S \rightarrow aAd \rightarrow 1$

$S \rightarrow \cdot bBd \rightarrow 2$

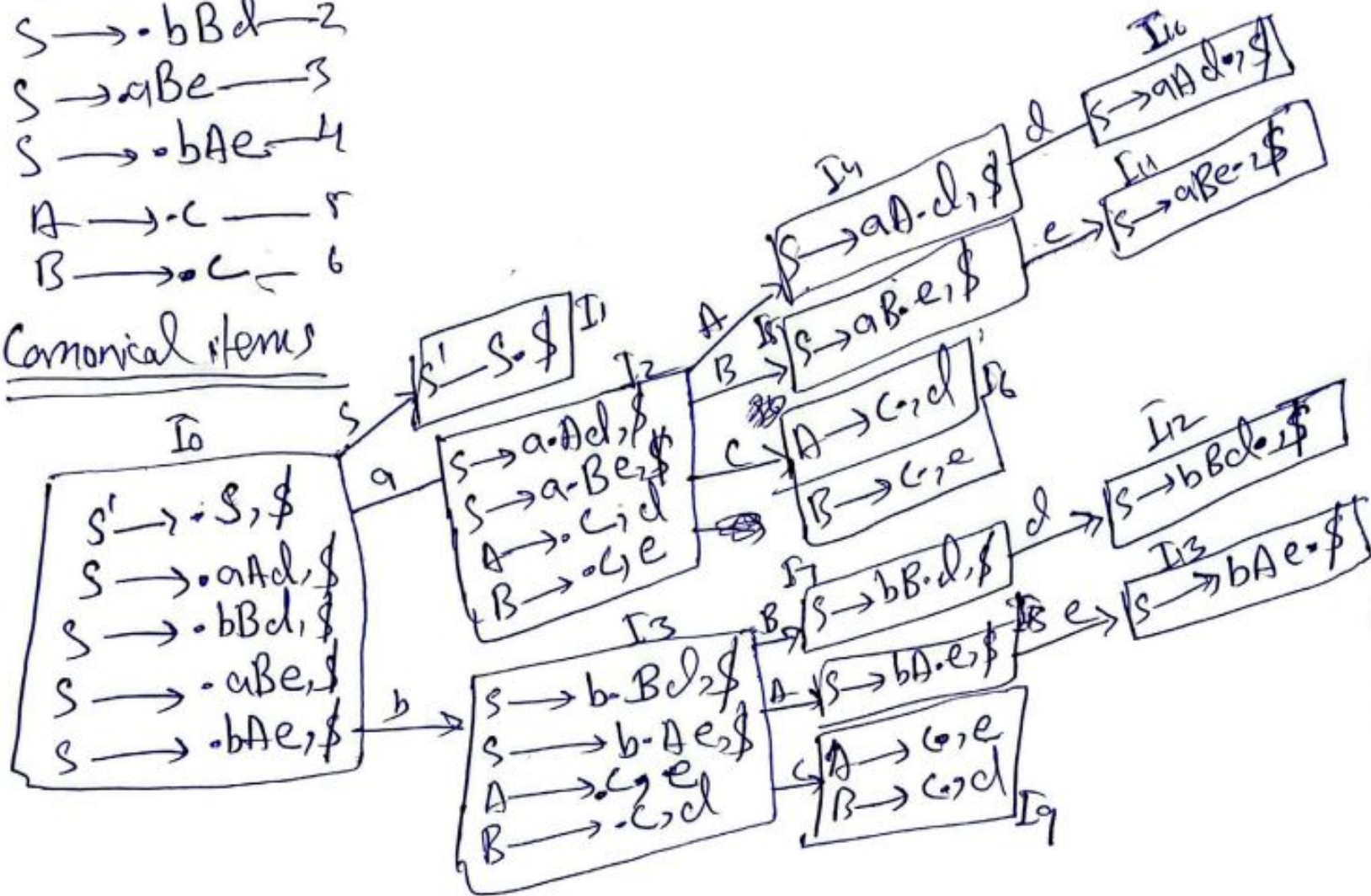
$S \rightarrow aBe \rightarrow 3$

$S \rightarrow \cdot bAe \rightarrow 4$

$A \rightarrow \cdot c \rightarrow 5$

$B \rightarrow \cdot c \rightarrow 6$

Canonical items



States	Actions			des		1 goto			
	a	b	c	d	e	\$	S	A	B
I ₀	S2	S3					1		
I ₁					Accept				
I ₂			S6					4	5
I ₃			S9					8	7
I ₄				S10	S11				
I ₅					S11				
I ₆				R5	R6				
I ₇				S12					
I ₈					S13				
I ₉				R6	R5				
I ₁₀						R1			
I ₁₁						R2			
I ₁₂						R2			
I ₁₃						R4			

LALR(1) Parser

- LALR Parser is lookahead LR parser. It is the most powerful parser which can handle large classes of grammar. The size of CLR parsing table is quite large as compared to other parsing table. LALR reduces the size of this table. LALR works similar to CLR. The only difference is , it combines the similar states of CLR parsing table into one single state.
- The general syntax becomes $[A \rightarrow \alpha . B, a]$ where $A \rightarrow \alpha . B$ is production and a is a terminal or right end marker \$
LR(1) items = LR(0) items + look ahead

How to add lookahead with the production?

- **CASE 1 –**

- $A \rightarrow \infty . BC, a$

- $B \rightarrow . D$

- Suppose this is B's production. The look ahead of this production is given as- we look at previous production i.e. - 0th production. Whatever is after B, we find FIRST(of that value) , that is the lookahead of 1st production. So, here in 0th production, after B, C is there. Assume $\text{FIRST}(C)=d$, then 1st production become.

- $B \rightarrow . D, d$

How to add lookahead with the production?

- **CASE 2 –**

- $A \rightarrow \infty . B, a$
- Here, we can see there's nothing after B. So the lookahead of 0th production will be the lookahead of 1st production. I.e
- $B \rightarrow . D, a$

How to add lookahead with the production?

- CASE 3 –
 - Assume a production $A \rightarrow a \mid b$
 - $A \rightarrow a, \$$ [0th production]
 - $A \rightarrow b, \$$ [1st production]
- Here, the 1st production is a part of the previous production, so the lookahead will be the same as that of its previous production.

Steps for constructing the LALR parsing table :

- Writing augmented grammar
- LR(1) collection of items to be found
- Defining 2 functions: goto[list of terminals] and action[list of non-terminals] in the LALR parsing table

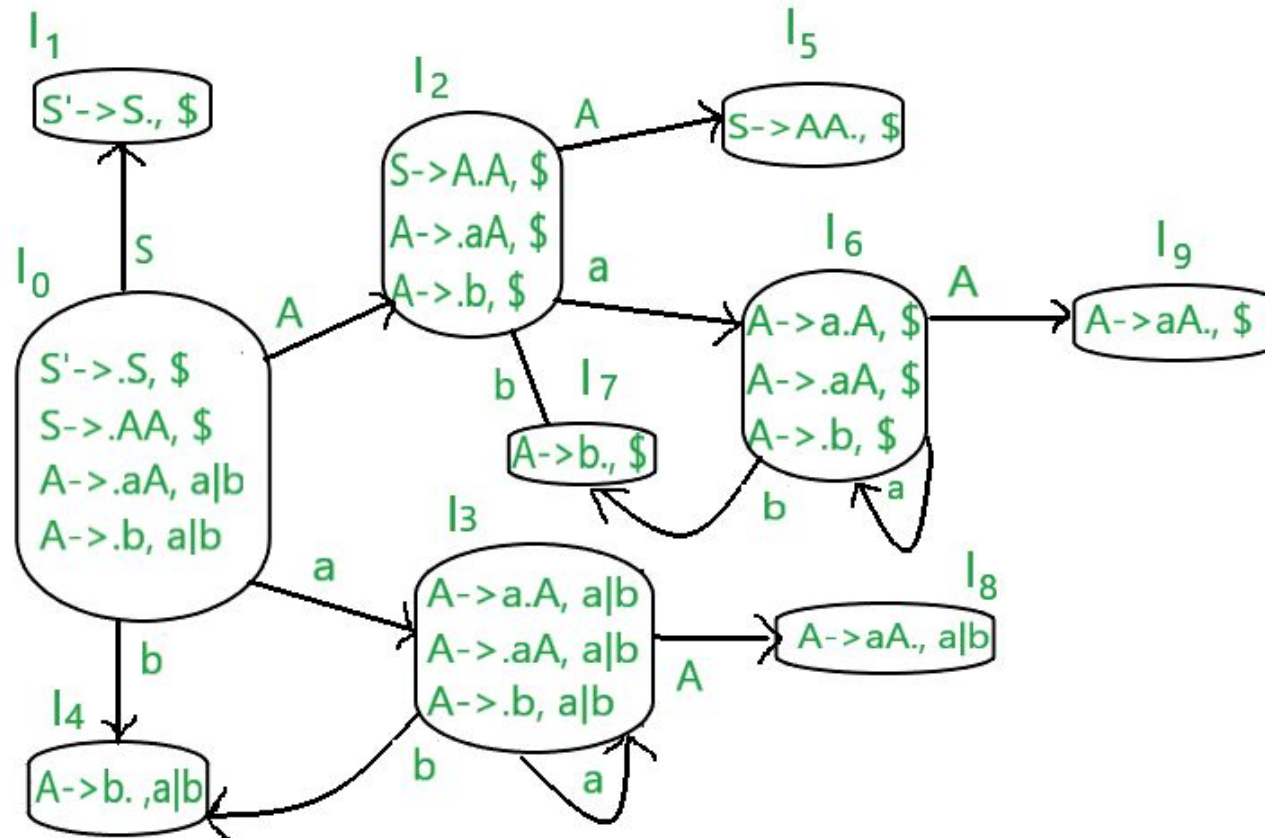
EXAMPLE

- $S \rightarrow AA$
- $A \rightarrow aA \mid b$
- Solution:
- **STEP1- Find augmented grammar**
 - The augmented grammar of the given grammar is:-
 - $S' \rightarrow .S, \$$ [0th production]
 - $S \rightarrow .AA, \$$ [1st production]
 - $A \rightarrow .aA, a \mid b$ [2nd production]
 - $A \rightarrow .b, a \mid b$ [3rd production]

Applying lookahead rule to the above productions.

- The initial look ahead is always \$
- Now, the 1st production came into existence because of ' . ' before 'S' in 0th production. There is nothing after 'S', so the lookahead of 0th production will be the lookahead of 1st production. i.e. : $S \rightarrow \cdot AA, \$$
- Now, the 2nd production came into existence because of ' . ' before 'A' in the 1st production.
- After 'A', there's 'A'. So, $FIRST(A)$ is a,b. Therefore, the lookahead of the 2nd production becomes a|b.
- Now, the 3rd production is a part of the 2nd production. So, the look ahead will be the same.

STEP2 - Find LR(0) collection of items



LALR Parsing Table

- The terminals of this grammar are $\{a,b\}$
- The non-terminals of this grammar are $\{S,A\}$

RULES

- If any non-terminal has ' . ' preceding it, we have to write all its production and add ' . ' preceding each of its production.
- from each state to the next state, the ' . ' shifts to one place to the right.
- In the figure, I0 consists of augmented grammar.
- I0 goes to I1 when ' . ' of 0th production is shifted towards the right of $S(S' \rightarrow S.)$. This state is the accept state. S is seen by the compiler. Since I1 is a part of the 0th production, the lookahead is same i.e. \$
- I0 goes to I2 when ' . ' of 1st production is shifted towards right $(S \rightarrow A.A)$. A is seen by the compiler. Since I2 is a part of the 1st production, the lookahead is same i.e. \$.
- I0 goes to I3 when ' . ' of 2nd production is shifted towards the right $(A \rightarrow a.A)$. a is seen by the compiler. since I3 is a part of 2nd production, the lookahead is same i.e. a|b.
- I0 goes to I4 when ' . ' of 3rd production is shifted towards right $(A \rightarrow b.)$. b is seen by the compiler. Since I4 is a part of 3rd production, the lookahead is same i.e. a|b.
- I2 goes to I5 when ' . ' of 1st production is shifted towards right $(S \rightarrow AA.)$. A is seen by the compiler. Since I5 is a part of the 1st production, the lookahead is same i.e. \$.

RULES (Continued)

- I2 goes to I6 when ' . ' of 2nd production is shifted towards the right ($A \rightarrow a.A$) . A is seen by the compiler. Since I6 is a part of the 2nd production, the lookahead is same i.e. \$.
- I2 goes to I7 when ' . ' of 3rd production is shifted towards right ($A \rightarrow b.$) . A is seen by the compiler. Since I6 is a part of the 3rd production, the lookahead is same i.e. \$.
- I3 goes to I3 when ' . ' of the 2nd production is shifted towards right ($A \rightarrow a.A$) . a is seen by the compiler. Since I3 is a part of the 2nd production, the lookahead is same i.e. a|b.
- I3 goes to I8 when ' . ' of 2nd production is shifted towards the right ($A \rightarrow aA.$) . A is seen by the compiler. Since I8 is a part of the 2nd production, the lookahead is same i.e. a|b.
- I6 goes to I9 when ' . ' of 2nd production is shifted towards the right ($A \rightarrow aA.$) . A is seen by the compiler. Since I9 is a part of the 2nd production, the lookahead is same i.e. \$.
- I6 goes to I6 when ' . ' of the 2nd production is shifted towards right ($A \rightarrow a.A$) . a is seen by the compiler. Since I6 is a part of the 2nd production, the lookahead is same i.e. \$.
- I6 goes to I7 when ' . ' of the 3rd production is shifted towards right ($A \rightarrow b.$) . b is seen by the compiler. Since I6 is a part of the 3rd production, the lookahead is same i.e. \$.

STEP 3 -

- Defining 2 functions: goto[list of terminals] and action[list of non-terminals] in the parsing table. Below is the **CLR parsing table**.

•

0 1 2 3 4 5 6 7 8 9	ACTION			GOTO	
	a	b	\$	A	S
	S3	S4		2	1
			accept		
	S6	S7		5	
	S3	S4		8	
	R3	R3			
			R1		
	S6	S7		9	
			R3		
	R2	R2			
		R2			

Making of LALR Parsing Table

- Once we make a CLR parsing table, we can easily make a LALR parsing table from it.
- In the **step2 diagram**, we can see that
 - I3 and I6 are similar except their lookaheads.
 - I4 and I7 are similar except their lookaheads.
 - I8 and I9 are similar except their lookaheads.
- In LALR parsing table construction , we merge these similar states.
 - Wherever there is 3 or 6, make it 36(combined form)
 - Wherever there is 4 or 7, make it 47(combined form)
 - Wherever there is 8 or 9, make it 89(combined form)

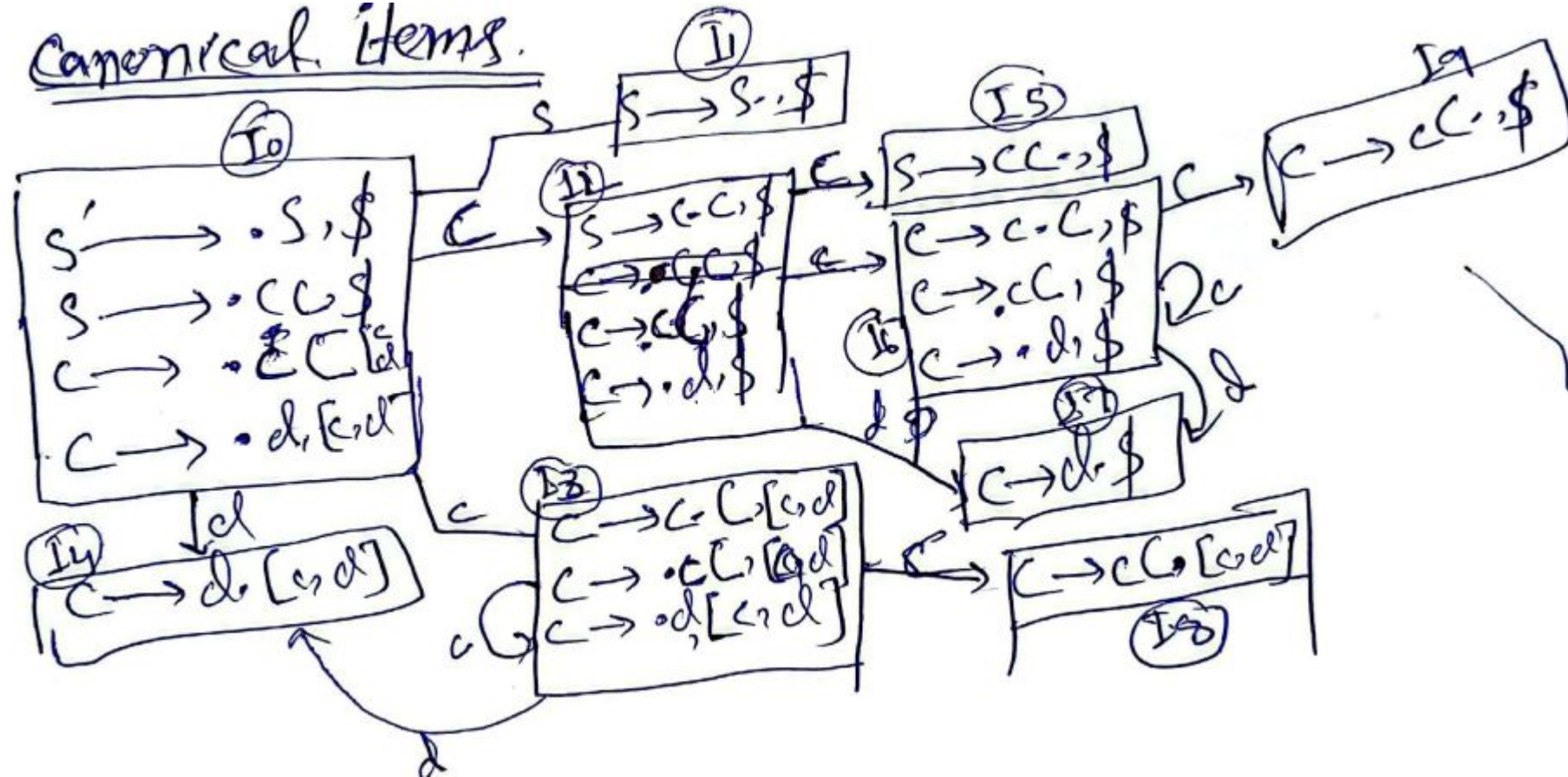
Below is the LALR parsing table.

	ACTION			GOTO	
	a	b	\$	A	S
0	S36	S47		2	1
1			accept		
2	S36	S47		5	
36	S36	S47		89	
47	R3	R3			
5			R1		
36	S36	S47		89	
47			R3		
89	R2	R2			
89			R2		

LALR Examples

<u>LALR Parser</u>	<u>Augmented Grammar</u>
Grammar	
$S \rightarrow CC$	$S' \rightarrow \cdot S \$ \quad \text{--- } 0$
$C \rightarrow cC$	$S \rightarrow \cdot CC \quad \text{--- } 1$
$C \rightarrow d.$	$C \rightarrow \cdot cC \quad \text{--- } 2$
	$C \rightarrow \cdot d. \quad \text{--- } 3$

Canonical Items.



LR Table Making it LALR

LALR. Parsing Table.

States	Actions			Goto	
	c	d	\$	s	c
I ₀	S3	S4		1	2
I ₁			Accept		
I ₂	S6	S7		3	5
I ₃	S3	S4			8
→ I ₄	R3	R3			
⊕ I ₅			R1		
I ₆	S6	S7			9
→ I ₇			R3		
I ₈	R2	R2			
⊖ I ₉			R2		

Making it LALR.

States	Actions			Goto	
	c	d	\$	s	c
I ₀	S36	S47		1	2
I ₁			Accept		
I ₂	S36	S47			5
I ₃₆	S36	S47			9
I ₄₇	R3	R3	R3		
I ₅			R1		
I ₈₉	R2	R2	R2		

- Now we have to remove the unwanted rows
- As we can see, 36 row has same data twice, so we delete 1 row.
- We combine two 47 row into one by combining each value in the single 47 row.
- We combine two 89 row into one by combining each value in the single 89 row.

The final LALR table looks like the below.

0 1 2 36 47 5 89	ACTION			GOTO	
	a	b	\$	A	S
	S36	S47		2	1
			accept		
	S36	S47		5	
	S36	S47		89	
	R3	R3	R3		
			R1		
	R2	R2	R2		

Reference:

- Compilers: Principles, Techniques, and Tools 2nd Edition
by Alfred Aho , Jeffrey Ullman, Ravi Sethi, Monica Lam
- Introduction to Compilers and Language Design | Second Edition |
Prof. Douglas Thain - University of Notre Dame